TipJar: A Secure Solana Creator Economy Platform Using MPC-Assisted Threshold Signing, Atomic Revenue Splits, and Integrated Staking

Author 1 Department / Institution City, Country email@example.com
Author 2 Department / Institution City, Country email@example.com

Abstract:
This paper presents TipJar, a full-stack prototype for creator monetization on Solana that combines wallet management, creator tipping, revenue-sharing, and staking within a unified application architecture. The project addresses a practical limitation in creator economy platforms: funds are often controlled by single-key wallets, payout splitting is handled off-chain or manually, and users are forced to move across disconnected interfaces for support, treasury management, and yield participation. TipJar proposes a repository-validated design in which wallet provisioning and transaction authorization are mediated through a multi-party computation (MPC) inspired threshold-signing workflow, while creator payouts are executed as atomic multi-recipient Solana transfers. The system is implemented as a Turborepo monorepo with a Next.js frontend, Express-based backend services, Prisma-managed PostgreSQL databases, and a reusable TypeScript package for Solana MPC/TSS operations. In addition, the frontend integrates a staking terminal that interacts with an Anchor-compatible staking program, allowing the same user environment to support both creator payments and asset deployment. Our analysis shows that the project contributes a strong application-level orchestration pattern for secure creator payments, especially through lamport-precise split calculation, wallet abstraction, and unified creator treasury workflows. At the same time, the current repository should be understood as a prototype rather than a production-grade threshold cryptography deployment, because several cryptographic and operational components remain simplified when compared with the threshold-signature and distributed key generation literature [1]-[4]. The paper therefore contributes both a system design and a critical implementation analysis suitable for future academic or product refinement.

Index Terms:
Solana, creator economy, multi-party computation, threshold signatures, wallet security, atomic payouts, staking, Web3 applications

I. INTRODUCTION

Blockchain-based creator platforms promise transparent payments, direct audience support, and programmable revenue distribution. In practice, however, most implementations still rely on single private keys, manual payout coordination, and fragmented user experiences. These gaps reduce trust and usability, especially for creators who collaborate with managers, editors, and co-producers that expect automated and verifiable revenue distribution. From the cryptographic side, decades of work in secret sharing, threshold signatures, and distributed key generation have shown that key authority can be distributed rather than concentrated in a single endpoint [1]-[4]. From the systems side, high-throughput blockchains such as Solana provide atomic transaction execution and relatively low-cost state transitions, making them attractive for creator-economy workflows that depend on frequent transfers and on-chain auditability [6], [7].

TipJar was designed to address these issues through a unified Solana application that supports four high-value workflows: secure wallet creation, creator tipping, multi-recipient payout distribution, and staking-based asset deployment. Instead of exposing a single wallet key to one service, the system separates wallet responsibility across backend and MPC-oriented services. Instead of transferring a tip to only one recipient and reconciling splits later, the platform calculates a platform fee and collaborator shares

in lamports, then constructs a single atomic on-chain transaction. Instead of forcing users to switch platforms for treasury activity, the application also exposes a staking dashboard tied to an Anchor-compatible program interface.

The central contribution of this work is not merely the existence of these features, but the way they are composed into a coherent, developer-operable architecture. The repository shows a clear attempt to combine application security, creator monetization logic, and Web3 usability in one system. This paper analyzes that design, explains its workflow, evaluates its prototype maturity, situates the prototype within prior literature, and identifies the changes required to move from hackathon-quality innovation toward publishable and deployable infrastructure.

II. PROBLEM STATEMENT AND MOTIVATION

The target problem is the secure management of creator-directed payments in a decentralized environment. Conventional creator platforms suffer from at least three shortcomings.

First, custodial or single-key wallet models introduce key concentration risk. If one server, device, or operator is compromised, assets become vulnerable. Second, creator teams frequently require split payments among multiple parties, but many systems handle these splits off-chain, increasing accounting overhead and trust assumptions. Third, user journeys across discovery, payment, wallet management, and staking are often fragmented, weakening adoption even when the underlying blockchain capabilities are strong.

TipJar responds by treating creator support as a systems problem rather than only a payment problem. The project attempts to secure wallet authority, minimize payout friction, preserve on-chain transparency, and offer a modern interface that lowers cognitive overhead for creators and supporters.

III. RELATED WORK AND LITERATURE CONTEXT

TipJar sits at the intersection of four research and engineering threads: threshold cryptography, blockchain transaction systems, application-layer security, and creator-economy platform design.

The first thread is threshold cryptography. Shamir's seminal secret-sharing construction showed how authority over a secret can be distributed across multiple participants, removing single points of compromise [1]. Later work on robust threshold DSS and secure distributed key generation extended this line into practical distributed signing systems and more secure initialization procedures for discrete-log-based cryptosystems [2], [3]. More recently, FROST demonstrated that threshold signing can be made significantly more communication-efficient while retaining strong security goals, making modern threshold approaches more attractive for networked applications [4]. TipJar does not yet implement a formally secure FROST-compatible workflow, but its architecture is clearly motivated by the same design goal: avoiding single-key custody.

The second thread is blockchain execution. Solana's Proof of History design and transaction model emphasize high-throughput, ordered, and atomic execution semantics, properties that are especially relevant to creator monetization because a single support action may need to trigger platform fees, collaborator payouts, state updates, and user-facing confirmation in one transaction boundary [6], [7]. Solana's verified build guidance also points toward a broader supply-chain security culture in which deployed on-chain binaries should be auditable against source artifacts [8]. This matters for any future version of TipJar that aims to migrate from prototype-level trust to stronger reproducibility guarantees.

The third thread is API and application security. Although TipJar is a blockchain application, its operational reality is still backend-heavy: user creation, profile management, revenue-split configuration, and transaction orchestration are exposed through APIs. The OWASP API Security Top 10 highlights recurring risks such as broken authentication, authorization flaws, and unsafe business logic exposure, all of which are directly relevant to wallet orchestration services [9]. Consequently, evaluating TipJar only as a smart-contract application would be incomplete;

it must also be assessed as an API-coordinated distributed system.

The fourth thread is creator-economy research. Recent scholarly work describes the creator economy as a rapidly growing and structurally distinct ecosystem in which creators, platforms, firms, and consumers interact through new value-creation and monetization models [10]. At the same time, Web3 research notes that decentralized infrastructure does not automatically eliminate value extraction or platform concentration at the application layer [11], [12]. TipJar can therefore be read as both an engineering prototype and an institutional experiment: it asks whether cryptographic coordination and on-chain settlement can reduce dependence on centralized monetization intermediaries without recreating new centralization bottlenecks at the orchestration layer.

This literature context helps frame the actual contribution of the project. TipJar is not proposing a new cryptographic primitive. Rather, it translates existing ideas from threshold key management, atomic blockchain settlement, and creator monetization into a concrete full-stack architecture that can be inspected, critiqued, and evolved.

IV. SYSTEM ARCHITECTURE

The repository is organized as a monorepo with multiple apps and shared packages. This separation is important because it maps cleanly onto the paper's system model.

A. Frontend Layer

The frontend is implemented in Next.js and provides authenticated user flows for landing, sign-in, creator discovery, creator profile exploration, tipping, wallet management, transaction history, and staking. The interface positions secure wallet operations behind a user-friendly dashboard abstraction rather than exposing low-level transaction tooling. This is especially visible in the creator dashboard, wallet page, and staking terminal, which collectively treat Web3 actions as application tasks instead of raw cryptographic events.

B. Application Backend

The primary backend is an Express service that manages authentication, profile operations, creator listing, tip orchestration, wallet inspection, transaction history retrieval, and administrative creator setup. It also computes payout distributions and coordinates multi-step signing with MPC servers. JWT-based middleware separates regular user access from admin routes.

C. MPC-Oriented Backend Node

A separate backend service stores one key share per user and participates in signing workflows. During user creation, this service generates a participant keypair and persists the corresponding share. During transfer or tipping requests, it produces step-one nonce material and step-two partial signatures for the parent service to aggregate.

D. Shared Libraries and Data Layer

The monorepo includes shared validation schemas, Solana configuration, database packages, and a reusable `solana-mpc-tss` package. Prisma is used for the main application database and the separate key-share database. This split is architecturally meaningful because it reduces direct coupling between user metadata and MPC share storage. The main database stores user identities, creator roles, tip history, and revenue split rules, while the MPC database stores key-share material required for signing participation.

V. METHODOLOGY AND TRANSACTION WORKFLOW

This section describes how TipJar implements its principal workflows.

A. Wallet Provisioning

Administrative user creation initiates wallet provisioning. The backend creates a user record, then sends a `create-user` request to the configured MPC server set. Each node generates a participant keypair and stores a secret share. The backend aggregates the returned public keys into a single user-facing public key and stores it in the primary user record. In the current repository, a devnet airdrop is attempted after wallet setup to simplify testing and onboarding.

This flow is valuable because it embeds wallet creation into identity provisioning. Users do not separately create and import wallets; the platform provisions a secure-feeling wallet abstraction as part of account lifecycle management.

B. Atomic Tipping With Revenue Splits

The most technically interesting workflow in the project is creator tipping. When a fan submits a tip, the backend retrieves the sender, the target creator, and any collaborator split rules. The total amount is converted into lamports, the platform fee is deducted, and remaining funds are distributed across collaborators and the creator. Importantly, the code explicitly handles rounding dust by assigning the remainder to the creator. This prevents inconsistent arithmetic and strengthens accounting determinism.

After payout computation, the backend initiates a two-step signing workflow across MPC servers. In step one, each server creates nonce-related data for the same transaction context. In step two, each server generates a partial signature using its stored share and the aggregated public nonce set. The backend then aggregates the returned signatures and broadcasts one multi-recipient Solana transaction. Finally, a tip record with the resulting transaction signature is persisted in the database.

This design is significant because it achieves platform fee extraction, collaborator compensation, and creator payment in a single on-chain action. Compared with manual post-settlement sharing, this approach improves transparency, reduces operational latency, and removes a major trust bottleneck.

C. General Wallet Transfer

The wallet page exposes a general transfer flow using the same two-step signing model. This demonstrates that the system is not limited to creator tipping; rather, tipping is one domain-specific transaction pattern built on top of a broader wallet orchestration layer.

D. Integrated Staking

TipJar also includes a staking terminal on the frontend. The staking hooks interact with an Anchor-compatible program interface to fetch pool state, user stake state, reward balances, and to submit stake, unstake, withdraw, and reward-claim actions. This broadens the platform from a payment application into a lightweight treasury interface, allowing users to hold, transfer, and deploy assets from one environment.

E. Repository-Centered Research Method

The methodology used in this paper is implementation-centered rather than simulation-centered. Instead of evaluating an abstract protocol in isolation, we analyze the repository as the primary artifact. This includes route inspection, package structure analysis, schema review, transaction-flow tracing, and local test execution. Such a methodology is appropriate for software-engineering research on prototypes because the central question is not only whether a protocol is secure in theory, but whether an end-to-end system composes security, usability, and maintainability into a coherent application workflow.

This approach also reveals a practical research advantage: discrepancies between declared architecture and implemented behavior become visible. For example, the backend presents a threshold-signing narrative at the application level, while the underlying signing library still contains simplified aggregation logic. That divergence is itself a meaningful result, because it identifies where future work must move from architectural aspiration to cryptographic rigor.

VI. PROTOTYPE EVALUATION

The present work evaluates TipJar as an implementation-backed prototype rather than as a completed production deployment. The evaluation is therefore based on repository analysis, architectural inspection, and available unit tests.

A. Functional Coverage

The repository demonstrates end-to-end support for:

- creator discovery and profile viewing;
- JWT-protected authentication and profile management;
- MPC-assisted wallet provisioning;
- atomic creator tipping with platform fee and collaborator splits;

- wallet balance and transaction lookup;
- general SOL transfer flows; and
- frontend staking interactions through an Anchor-compatible IDL.

This breadth is notable because many student or hackathon systems implement only one of these flows in isolation.

B. Test Evidence

The `packages/solana-mpc-tss` package contains local Jest tests for MPC signing, keypair behavior, transaction creation, key aggregation, recent blockhash retrieval, airdrop logic, aggregate signing steps, and insufficient-signature handling. A local test run on March 29, 2026 completed with **29 passing tests out of 30**. The single failing test expects a 32-byte secret key, while the current wallet implementation returns a 64-byte secret key from the Solana keypair generator. This result suggests that the package is actively exercised, but still has interface consistency issues that should be resolved before claiming production maturity.

C. Design Strengths

The prototype exhibits several strong design decisions:

- Monorepo modularity cleanly separates UI, orchestration, cryptographic logic, and persistence.
- Lamport-based payout arithmetic reduces floating-point ambiguity in payment distribution.
- Revenue split automation is implemented directly at the transaction-construction layer.
- A separate MPC-oriented service makes wallet operations easier to reason about operationally.
- Shared schemas and typed packages improve consistency across services.

D. Evaluation Limitations

Because TipJar is currently evaluated through repository inspection and local tests, this paper does not claim benchmark-grade measurements for throughput, latency, or adversarial fault tolerance. The available test evidence is strongest at the package level rather than at the distributed deployment level. In particular, the current project does not yet present formal empirical results for multi-node signing latency, sustained request concurrency, or economically realistic creator-payment workloads. For a mature conference submission, those measurements would significantly strengthen the evidence base.

Nevertheless, the present evaluation remains useful because it addresses a different but still important question: whether the implementation meaningfully embodies the design ideas it claims to represent. In that respect, the answer is yes. The platform already demonstrates a credible orchestration model for creator payments, even if the underlying cryptographic machinery is not yet fully production-ready.

VII. SECURITY ANALYSIS

From a research perspective, TipJar is most compelling when read as a secure application architecture prototype rather than a finished threshold cryptography implementation.

A. Positive Security Properties

The repository clearly attempts to avoid direct single-key exposure at the application workflow level. Wallet provisioning and transaction authorization are mediated through separate services, and key-share material is stored outside the main application database. Administrative and user JWT flows are separated, and transaction settlement is recorded with on-chain signatures. These choices improve traceability and make the system easier to evolve toward stronger custody models.

The use of atomic multi-recipient transfers also provides a subtle security and integrity benefit. By settling platform fees, collaborator shares, and creator payouts in one transaction, the design reduces temporal gaps in which downstream accounting or payout logic could fail, be manipulated, or drift from user expectation. On Solana, transaction atomicity means either the entire payout plan succeeds or the operation is rolled back [7]. For creator-economy systems, that property is not merely a performance convenience; it is a trust mechanism.

B. Threat Model

The system can be interpreted under a practical threat model involving at least five classes of risk.

First, there is single-node compromise risk. If a single service holds complete signing authority, compromise directly implies asset compromise. TipJar's intended architecture attempts to reduce this risk by splitting responsibility across an application backend and MPC-oriented services.

Second, there is orchestration risk. Even if the cryptographic core is sound, backend logic may calculate wrong distributions, submit incorrect recipients, or mishandle authorization checks. TipJar partly addresses this through shared schemas and a centralized transaction-construction path, but business-logic testing remains an area for further strengthening.

Third, there is credential and API risk. Because user login and administrative control flow through ordinary web APIs, the platform must still defend against the classes of vulnerabilities highlighted by modern API-security guidance, particularly authorization and authentication weaknesses [9].

Fourth, there is storage risk. Persisted signing shares, if not encrypted and operationally isolated, become a high-value target even if the signing protocol is threshold-aware in design.

Fifth, there is supply-chain and deployment risk. Smart-contract and backend trust both depend on reproducible deployment practices, dependency integrity, and binary verification. Solana's verified-build guidance is therefore relevant to the long-term maturation of the project [8].

C. Prototype Limitations

Several findings prevent the current implementation from being presented as production-grade MPC/TSS:

1. The configured MPC server list currently contains only one node in the backend route configuration. In practice, this makes the active threshold behavior effectively single-node.
2. Public-key aggregation and signature aggregation in the TSS package are implemented with simplified XOR-based logic, which is not a substitute for a formally secure threshold signature protocol.
3. The wallet key generation path currently falls back to ordinary Solana keypair creation.
4. User passwords are compared directly rather than being hashed and salted.
5. Secret key shares are persisted as plain strings in the MPC database layer, which would require encryption at rest, key wrapping, or HSM-backed storage in a real deployment.

These limitations do not eliminate the system's research value. Instead, they define the paper's honest contribution: TipJar is a strong application-level framework for secure creator payment orchestration, with a clear roadmap toward rigorous cryptographic hardening.

VIII. DEPLOYMENT AND SCALABILITY CONSIDERATIONS

Although the current repository is a prototype, its architecture already suggests how a larger deployment could evolve.

One deployment path would separate responsibilities into at least three operational zones: a public API tier, an internal signing-coordination tier, and multiple independently administered MPC or threshold-signing nodes. Such a model would better reflect the trust distribution expected of genuine threshold systems. It would also allow different observability, networking, and hardening policies for user-facing services versus key-sensitive services.

Scalability is also influenced by the decision to perform split computation off-chain and settlement on-chain. This is generally favorable. Off-chain computation allows flexible business logic and metadata handling, while on-chain settlement preserves atomicity and public verification. However, scaling such a model requires careful control over transaction size, signer coordination latency, and the number of recipients in a single payout transaction. Solana's transaction rules and message structure are

therefore directly relevant to future optimization efforts [7].

The frontend architecture is similarly extensible. Because creator discovery, tipping, wallet management, and staking are already modularized, the user experience can be expanded to support treasury analytics, role-based dashboards for collaborators, automated split templates, or risk notices around pending on-chain actions. In other words, the current frontend is not merely decorative; it is already organized as a potential control plane for a larger creator-finance system.

IX. DISCUSSION

TipJar demonstrates that secure creator economy infrastructure should be studied as an interaction between cryptography, application design, and user experience. Many academic proposals stop at the protocol layer, while many product prototypes stop at the interface layer. This repository is valuable because it bridges both domains. The user sees a coherent dashboard; the developer sees a layered architecture; and the researcher sees a concrete experiment in mapping threshold-signing ideas onto creator-finance workflows.

Another important contribution is the use of atomic multi-recipient transactions as a fairness mechanism. Revenue-sharing is not treated as a later bookkeeping exercise but as a first-class settlement primitive. For creator teams, this can reduce disputes, shorten payment cycles, and improve trust in platform logic.

The inclusion of staking also opens an interesting line of future work. A creator platform need not stop at tip acceptance. It can evolve into a programmable treasury surface where incoming support, retained balances, yield strategies, and collaborator payouts are all managed inside one policy-aware wallet environment. This is particularly important in the context of the broader creator-economy literature, which increasingly treats creators as small firms with multifaceted revenue, branding, and financial-management requirements rather than as isolated content producers [10]. TipJar's integrated design is therefore research-relevant not only because it changes settlement, but because it changes the operational unit of analysis from "tip" to "creator treasury".

At the same time, scholarship on Web3 warns that decentralization at the infrastructure layer does not guarantee decentralization at the application layer [11], [12]. TipJar reflects this tension clearly. It seeks to reduce centralized custody and manual payout dependence, yet its current orchestration logic still passes through central services. This does not invalidate the design. Instead, it sharpens the academic question: how much decentralization is necessary at each layer to meaningfully improve creator autonomy without sacrificing usability?

X. FUTURE WORK

To elevate TipJar from an impressive prototype to a publication-ready and deployment-ready system, future work should prioritize the following:

1. Replace the simplified signing flow with a formally secure threshold signature protocol such as FROST-style signing for Ed25519-compatible workflows.
2. Expand MPC deployment from one configured node to a real multi-node threshold environment such as 2-of-3 or 3-of-5.
3. Introduce password hashing, encrypted key-share storage, audit logging, rate limiting, and service authentication between backend nodes.
4. Benchmark signing latency, end-to-end transaction time, database performance, and user-facing responsiveness under concurrent load.
5. Evaluate creator-team satisfaction and perceived trust through user studies that compare manual payout systems with atomic split settlement.
6. Extend the staking subsystem with policy-aware treasury allocation and analytics for creators.

XI. CONCLUSION

TipJar presents a thoughtful and technically ambitious approach to creator monetization on Solana. Its strongest contribution lies in combining MPC-inspired wallet orchestration, atomic payout splitting, wallet management, and staking within one integrated product architecture. The repository already demonstrates meaningful system-level innovation, particularly in how creator tips are transformed into precise, multi-recipient on-chain settlement flows. Although the current cryptographic implementation remains prototype-grade, the architectural foundation is strong and research-relevant. As a result, TipJar is best understood as a high-potential secure creator-finance framework that can support both academic extension and real-world product hardening.

REFERENCES

[1] A. Shamir, "How to Share a Secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612-613, 1979, doi: 10.1145/359168.359176.

[2] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Robust Threshold DSS Signatures," in *Advances in Cryptology - EUROCRYPT '96*, 1996, pp. 354-371, doi: 10.1007/3-540-68339-9_31.

[3] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure Distributed Key Generation for Discrete-Log Based Cryptosystems," in *Advances in Cryptology - EUROCRYPT '99*, 1999, pp. 295-310, doi: 10.1007/3-540-48910-X_21.

[4] C. Komlo and I. Goldberg, "FROST: Flexible Round-Optimized Schnorr Threshold Signatures," *Cryptology ePrint Archive*, Paper 2020/852, 2020. [Online]. Available: https://eprint.iacr.org/2020/852

[5] S. Josefsson and I. Liusvaara, "RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)," Internet Engineering Task Force, Jan. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8032.html

[6] A. Yakovenko, "Proof of History: A Clock for Blockchain," Solana Foundation, Apr. 18, 2018. [Online]. Available: https://solana.com/en/news/proof-of-history--a-clock-for-blockchain

[7] Solana Foundation, "Transactions," Solana Documentation. [Online]. Available: https://solana.com/docs/core/transactions

[8] Solana Foundation, "Verifying Programs," Solana Documentation. [Online]. Available: https://solana.com/docs/programs/verified-builds

[9] OWASP Foundation, "OWASP API Security Top 10 - 2023," OWASP API Security Project. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x00-header/

[10] R. Peres, M. Schreier, D. A. Schweidel, and A. Sorescu, "The creator economy: An introduction and a call for scholarly research," *International Journal of Research in Marketing*, vol. 41, no. 3, pp. 403-410, 2024, doi: 10.1016/j.ijresmar.2024.07.005.

[11] D. Sarkar, "Centralized Intermediation in a Decentralized Web3 Economy: Value Accrual and Extraction," *arXiv preprint arXiv:2311.08234*, 2023. [Online]. Available: https://arxiv.org/abs/2311.08234

[12] P. P. Momtaz, "Some Very Simple Economics of Web3 and the Metaverse," *FinTech*, vol. 1, no. 3, pp. 225-234, 2022, doi: 10.3390/fintech1030018.

[13] TipJar monorepo, analyzed from local repository structure, frontend applications, backend services, Prisma schemas, and the `solana-mpc-tss` package.

[14] Anchor-compatible staking interface definitions included in the frontend IDL integration and local project sources.